

SVNIT Department of Artificial Intelligence

Subject: Machine Learning (AI301)

Assignment 3: Decision Tree for Heart Disease Prediction

Aim: To train and evaluate a Decision Tree classifier on the Heart Disease dataset, perform hyperparameter tuning, analyse misclassifications, and handle class imbalance.

Objectives:

1. Train a Decision Tree and report training/testing accuracy.
2. Visualize the decision tree and interpret the first two splits.
3. Evaluate the model using confusion matrix, classification report, ROC curve, and AUC score.
4. Tune hyperparameters using manual search and GridSearchCV.
5. Perform error analysis on misclassified patients.
6. Check class imbalance and apply corrective measures if required.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_curve, roc_auc_score, balanced_accuracy_score
)
from imblearn.over_sampling import SMOTE

# Visual style
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12

print('All libraries imported successfully.')
```

All libraries imported successfully.

```
In [2]: # Load Heart Disease dataset (extracted from heart_v2.pdf)
df = pd.read_csv('heart_v2.csv')

print(f'Dataset loaded: {df.shape[0]} rows, {df.shape[1]} columns')
```

```

print(f'Columns: {list(df.columns)}')
print(f'\nMissing values: {df.isnull().sum().sum()}')
print(f'\nDescriptive Statistics:')
print(df.describe().round(2))
print(f'\nFirst 10 rows:')
df.head(10)

```

Dataset loaded: 270 rows, 5 columns

Columns: ['age', 'sex', 'BP', 'cholesterol', 'heart disease']

Missing values: 0

Descriptive Statistics:

	age	sex	BP	cholesterol	heart disease
count	270.00	270.00	270.00	270.00	270.00
mean	54.43	0.68	131.34	249.66	0.44
std	9.11	0.47	17.86	51.69	0.50
min	29.00	0.00	94.00	126.00	0.00
25%	48.00	0.00	120.00	213.00	0.00
50%	55.00	1.00	130.00	245.00	0.00
75%	61.00	1.00	140.00	280.00	1.00
max	77.00	1.00	200.00	564.00	1.00

First 10 rows:

```

Out[2]:

```

	age	sex	BP	cholesterol	heart disease
0	70	1	130	322	1
1	67	0	115	564	0
2	57	1	124	261	1
3	64	1	128	263	0
4	74	0	120	269	0
5	65	1	120	177	0
6	56	1	130	256	1
7	59	1	110	239	1
8	60	1	140	293	1
9	63	0	150	407	1

Part A: Basic Implementation

Train a Decision Tree classifier on the Heart Disease dataset, report training and testing accuracy, visualize the tree, and interpret the first two splits.

```

In [3]: # A.1: Prepare features and target, split data
X = df[['age', 'sex', 'BP', 'cholesterol']]
y = df['heart disease']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'Train size : {len(X_train)}')
print(f'Test size : {len(X_test)}')

```

```
print(f'\nClass distribution in train:\n{y_train.value_counts().sort_index().to_s
print(f'\nClass distribution in test :\n{y_test.value_counts().sort_index().to_s
```

Train size : 216

Test size : 54

Class distribution in train:

heart disease

0 120

1 96

Class distribution in test :

heart disease

0 30

1 24

```
In [4]: # A.2: Train Decision Tree with default parameters
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

train_acc = accuracy_score(y_train, dt_model.predict(X_train))
test_acc = accuracy_score(y_test, dt_model.predict(X_test))

print('Decision Tree (default parameters)')
print('=' * 42)
print(f'Training Accuracy : {train_acc * 100:.2f}%')
print(f'Testing Accuracy : {test_acc * 100:.2f}%')
print(f'Tree Depth : {dt_model.get_depth()}')
print(f'Number of Leaves : {dt_model.get_n_leaves()}')

if train_acc - test_acc > 0.15:
    print('\nObservation: Large gap between train and test accuracy indicates O
    print('
        The model has memorized the training data too closely.')
else:
    print('\nObservation: Train and test accuracy are close – model generalizes
```

Decision Tree (default parameters)

=====

Training Accuracy : 100.00%

Testing Accuracy : 66.67%

Tree Depth : 11

Number of Leaves : 60

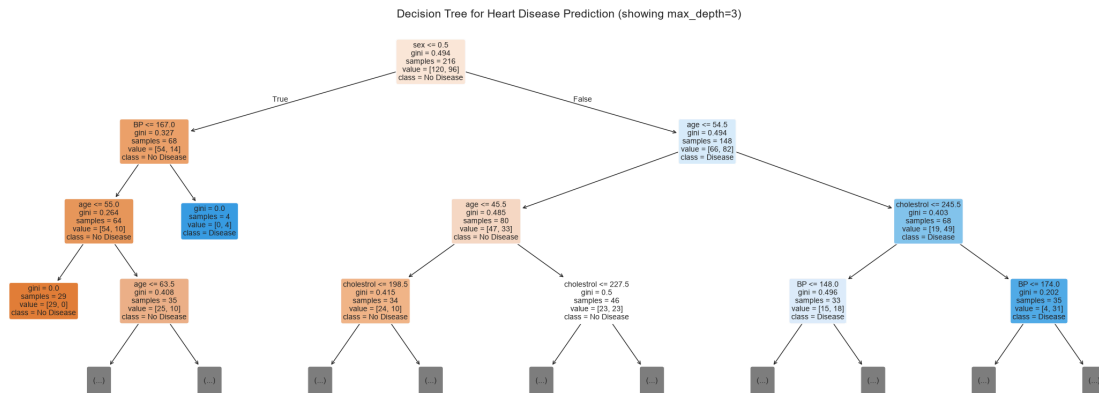
Observation: Large gap between train and test accuracy indicates OVERFITTING.

The model has memorized the training data too closely.

```
In [5]: # A.3: Visualize the Decision Tree (depth=3 for readability)
fig, ax = plt.subplots(figsize=(22, 8))
plot_tree(
    dt_model,
    max_depth=3,
    feature_names=X.columns.tolist(),
    class_names=['No Disease', 'Disease'],
    filled=True,
    rounded=True,
    fontsize=10,
    ax=ax
)
ax.set_title('Decision Tree for Heart Disease Prediction (showing max_depth=3)')
plt.tight_layout()
plt.show()
```

```
# A.4: Interpret first two splits
print('Text Representation (first 2 levels):')
print(export_text(dt_model, feature_names=X.columns.tolist(), max_depth=2))

root_feat = X.columns[dt_model.tree_.feature[0]]
root_thresh = dt_model.tree_.threshold[0]
print('Interpretation of the first two splits:')
print(f' Split 1 (Root): The model first splits on "{root_feat}" <= {root_thresh}')
print(f' -> "{root_feat}" has the highest information gain, making it the most')
print(f' discriminative feature for predicting heart disease at the root node.')
print(f' Split 2: Each child of the root further splits the data using the next most')
print(f' informative feature, progressively narrowing down the decision regions')
```



Text Representation (first 2 levels):

```
--- sex <= 0.50
|   |--- BP <= 167.00
|   |   |--- age <= 55.00
|   |   |   |--- class: 0
|   |   |   |--- age > 55.00
|   |   |   |   |--- truncated branch of depth 8
|   |   |--- BP > 167.00
|   |   |   |--- class: 1
|--- sex > 0.50
|   |--- age <= 54.50
|   |   |--- age <= 45.50
|   |   |   |--- truncated branch of depth 6
|   |   |   |--- age > 45.50
|   |   |   |   |--- truncated branch of depth 9
|   |   |--- age > 54.50
|   |   |   |--- cholesterol <= 245.50
|   |   |   |   |--- truncated branch of depth 8
|   |   |   |   |--- cholesterol > 245.50
|   |   |   |   |   |--- truncated branch of depth 6
```

Interpretation of the first two splits:

Split 1 (Root): The model first splits on "sex" <= 0.50.

-> "sex" has the highest information gain, making it the most discriminative feature for predicting heart disease at the root node.

Split 2: Each child of the root further splits the data using the next most informative feature, progressively narrowing down the decision regions.

Part B: Model Evaluation

Evaluate the Decision Tree using a confusion matrix, classification report (precision, recall, F1-score), ROC curve, and AUC score.

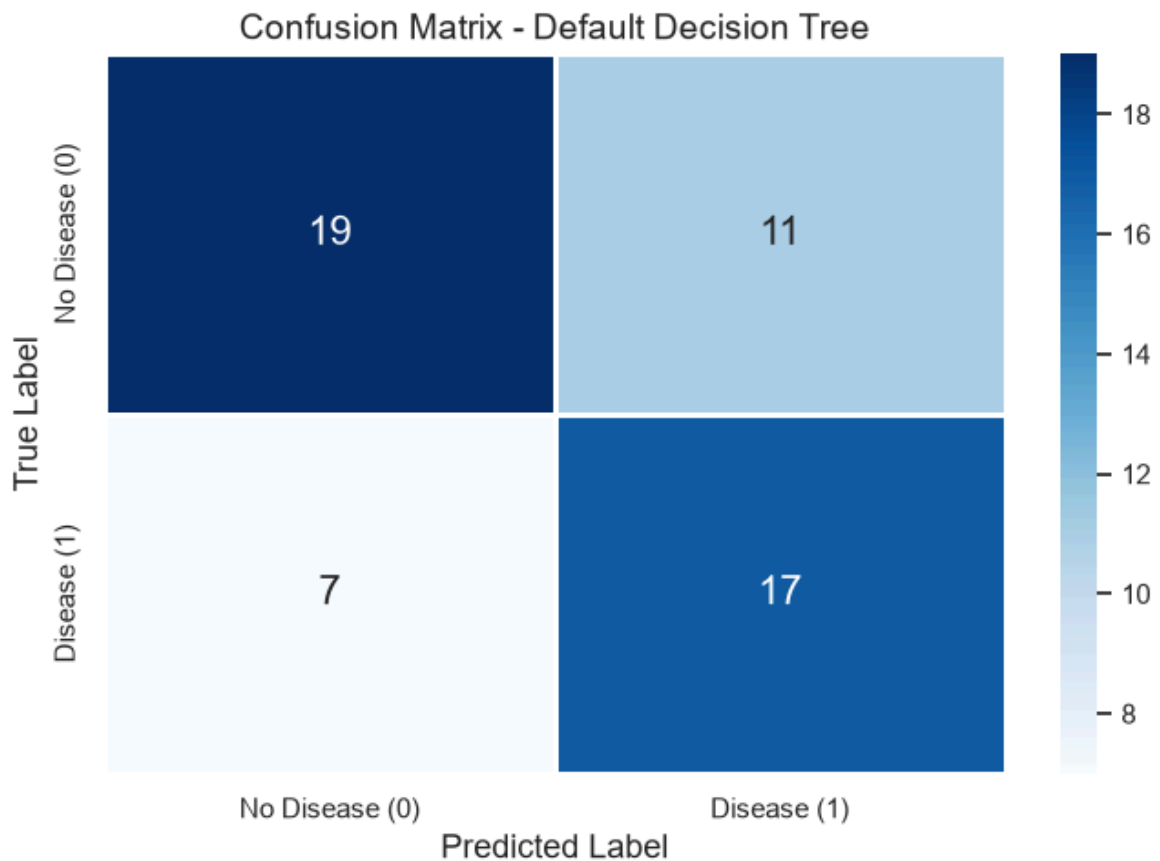
```

In [6]: # B.1: Confusion Matrix
y_pred = dt_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['No Disease (0)', 'Disease (1)'],
    yticklabels=['No Disease (0)', 'Disease (1)'],
    linewidths=1, annot_kws={'size': 16}
)
plt.xlabel('Predicted Label', fontsize=13)
plt.ylabel('True Label', fontsize=13)
plt.title('Confusion Matrix - Default Decision Tree')
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f'True Negatives (TN) - Correctly predicted No Disease : {tn}')
print(f'False Positives (FP) - Predicted Disease, Actually No : {fp}')
print(f'False Negatives (FN) - Predicted No Disease, Actually Disease : {fn} <-- Clinically Critical')
print(f'True Positives (TP) - Correctly predicted Disease : {tp}')

```



```

True Negatives (TN) - Correctly predicted No Disease : 19
False Positives (FP) - Predicted Disease, Actually No : 11
False Negatives (FN) - Predicted No Disease, Actually Disease : 7 <-- Clinically Critical
True Positives (TP) - Correctly predicted Disease : 17

```

```

In [7]: # B.2: Classification Report
print('Classification Report - Default Decision Tree')
print('=' * 55)
print(classification_report(y_test, y_pred, target_names=['No Disease', 'Disease']))

```

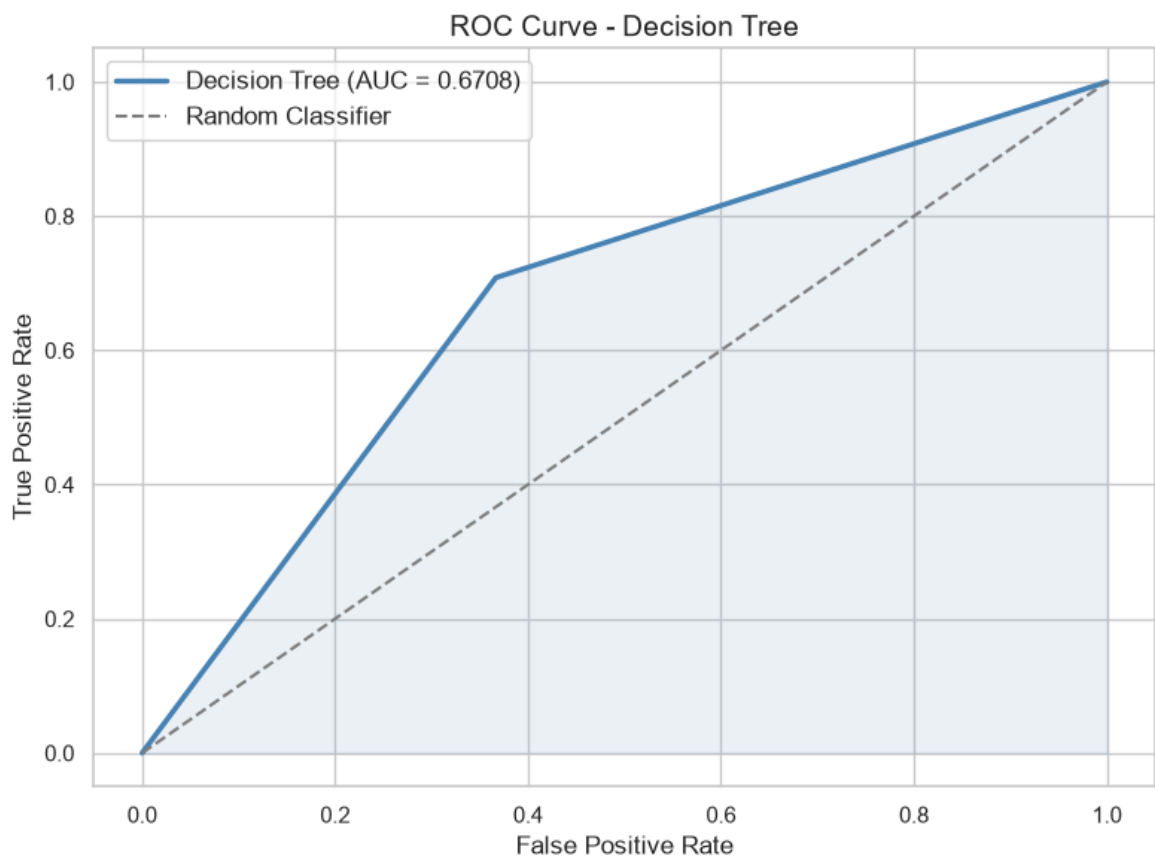
Classification Report - Default Decision Tree

	precision	recall	f1-score	support
No Disease	0.73	0.63	0.68	30
Disease	0.61	0.71	0.65	24
accuracy			0.67	54
macro avg	0.67	0.67	0.67	54
weighted avg	0.68	0.67	0.67	54

```
In [8]: # B.3: ROC Curve and AUC Score
y_prob = dt_model.predict_proba(X_test)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='steelblue', linewidth=2.5,
         label=f'Decision Tree (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', linewidth=1.5, label='Random')
plt.fill_between(fpr, tpr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Decision Tree')
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()

print(f'AUC Score: {auc_score:.6f}')
if auc_score >= 0.9:
    print('Interpretation: Excellent discrimination ability.')
elif auc_score >= 0.8:
    print('Interpretation: Good discrimination ability.')
elif auc_score >= 0.7:
    print('Interpretation: Fair discrimination ability.')
else:
    print('Interpretation: Poor - model needs improvement (hyperparameter tuning)')
```



AUC Score: 0.670833

Interpretation: Poor – model needs improvement (hyperparameter tuning may help).

Part C: Hyperparameter Tuning

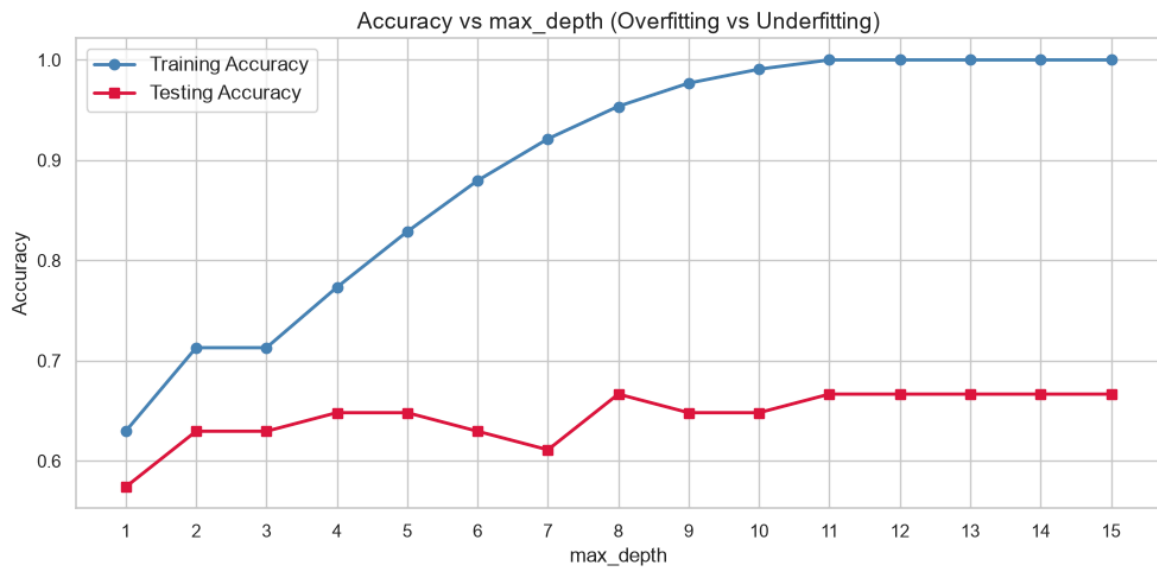
Vary `max_depth`, `min_samples_split`, and `min_samples_leaf` to analyze the overfitting vs underfitting tradeoff. Then use `GridSearchCV` for automated optimal hyperparameter search.

```
In [9]: # C.1: Vary max_depth
depths = list(range(1, 16))
train_accs_d, test_accs_d = [], []

for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train, y_train)
    train_accs_d.append(accuracy_score(y_train, m.predict(X_train)))
    test_accs_d.append(accuracy_score(y_test, m.predict(X_test)))

plt.figure(figsize=(10, 5))
plt.plot(depths, train_accs_d, marker='o', color='steelblue', linewidth=2, label='Train Accuracy')
plt.plot(depths, test_accs_d, marker='s', color='crimson', linewidth=2, label='Test Accuracy')
plt.xlabel('max_depth')
plt.ylabel('Accuracy')
plt.title('Accuracy vs max_depth (Overfitting vs Underfitting)')
plt.legend(fontsize=12)
plt.xticks(depths)
plt.tight_layout()
plt.show()

best_depth = depths[test_accs_d.index(max(test_accs_d))]
print(f'Best max_depth by test accuracy: {best_depth} (test acc = {max(test_accs_d)})')
print(f'Observation: Small depth = underfitting, Very large depth = overfitting')
```



Best max_depth by test accuracy: 8 (test acc = 66.67%)

Observation: Small depth = underfitting, Very large depth = overfitting (train 100%, test drops).

```
In [10]: # C.2: Vary min_samples_split and min_samples_leaf
splits = list(range(2, 30, 2))
tr_accs_s, te_accs_s = [], []
for s in splits:
    m = DecisionTreeClassifier(min_samples_split=s, random_state=42)
    m.fit(X_train, y_train)
    tr_accs_s.append(accuracy_score(y_train, m.predict(X_train)))
    te_accs_s.append(accuracy_score(y_test, m.predict(X_test)))

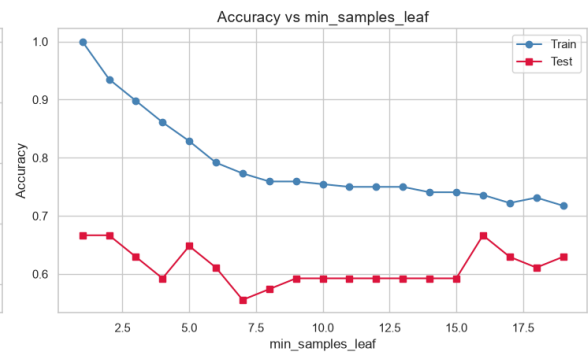
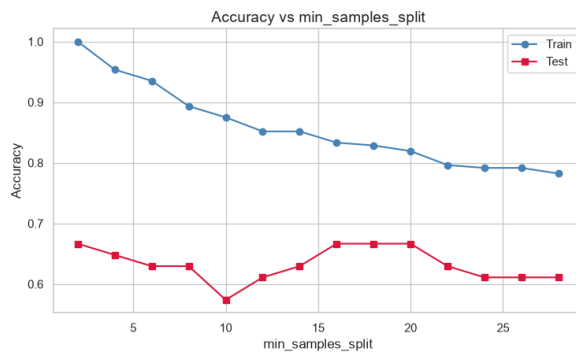
leaves = list(range(1, 20))
tr_accs_l, te_accs_l = [], []
for lv in leaves:
    m = DecisionTreeClassifier(min_samples_leaf=lv, random_state=42)
    m.fit(X_train, y_train)
    tr_accs_l.append(accuracy_score(y_train, m.predict(X_train)))
    te_accs_l.append(accuracy_score(y_test, m.predict(X_test)))

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(splits, tr_accs_s, marker='o', color='steelblue', label='Train')
axes[0].plot(splits, te_accs_s, marker='s', color='crimson', label='Test')
axes[0].set_xlabel('min_samples_split')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy vs min_samples_split')
axes[0].legend()

axes[1].plot(leaves, tr_accs_l, marker='o', color='steelblue', label='Train')
axes[1].plot(leaves, te_accs_l, marker='s', color='crimson', label='Test')
axes[1].set_xlabel('min_samples_leaf')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Accuracy vs min_samples_leaf')
axes[1].legend()

plt.suptitle('Hyperparameter Sensitivity Analysis', fontsize=14)
plt.tight_layout()
plt.show()
```



```
In [11]: # C.3: GridSearchCV for automated hyperparameter tuning
param_grid = {
    'max_depth'      : [2, 3, 4, 5, 6, None],
    'min_samples_split' : [2, 5, 10, 15],
    'min_samples_leaf'  : [1, 2, 4, 6],
    'criterion'        : ['gini', 'entropy'],
}

grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=0
)
grid_search.fit(X_train, y_train)

best_params      = grid_search.best_params_
best_cv_score     = grid_search.best_score_
best_dt          = grid_search.best_estimator_

tuned_train_acc  = accuracy_score(y_train, best_dt.predict(X_train))
tuned_test_acc   = accuracy_score(y_test, best_dt.predict(X_test))

print('GridSearchCV Results')
print('=' * 50)
print(f'Best Parameters      : {best_params}')
print(f'Best CV Accuracy      : {best_cv_score * 100:.2f}%')
print(f'\nTuned Model Performance:')
print(f'  Training Accuracy    : {tuned_train_acc * 100:.2f}%')
print(f'  Testing Accuracy     : {tuned_test_acc * 100:.2f}%')
print(f'\nImprovement over default test accuracy: {(tuned_test_acc - test_acc)*100:.2f}%')
```

GridSearchCV Results

=====

Best Parameters : {'criterion': 'gini', 'max_depth': 2, 'min_samples_leaf': 1, 'min_samples_split': 2}

Best CV Accuracy : 69.43%

Tuned Model Performance:

Training Accuracy : 71.30%

Testing Accuracy : 62.96%

Improvement over default test accuracy: -3.70%

Part D: Error Analysis

Identify patients misclassified by the tuned Decision Tree, compare their features against correctly classified patients, and extract patterns about what makes a patient harder to classify.

```
In [12]: # D.1: Identify misclassified patients
y_pred_tuned = best_dt.predict(X_test)

test_df = X_test.copy().reset_index(drop=True)
test_df['true_label'] = y_test.values
test_df['predicted_label'] = y_pred_tuned
test_df['correct'] = (test_df['true_label'] == test_df['predicted_label'])

misclassified = test_df[~test_df['correct']]
correct = test_df[test_df['correct']]

print(f'Total test samples : {len(test_df)}')
print(f'Correctly classified: {len(correct)} ({len(correct)/len(test_df)*100:.1f}%)')
print(f'Misclassified : {len(misclassified)} ({len(misclassified)/len(test_df)*100:.1f}%)')

# Breakdown of misclassification types
fn_mask = (test_df['true_label'] == 1) & (test_df['predicted_label'] == 0)
fp_mask = (test_df['true_label'] == 0) & (test_df['predicted_label'] == 1)
print(f'\nFalse Negatives (Disease missed) : {fn_mask.sum()} <-- Clinically most dangerous')
print(f'False Positives (Healthy misdiag): {fp_mask.sum()}')
print('\nMisclassified patients:')
print(misclassified.to_string(index=False))
```

```
Total test samples : 54
Correctly classified: 34 (63.0%)
Misclassified : 20 (37.0%)
```

```
False Negatives (Disease missed) : 11 <-- Clinically most dangerous
False Positives (Healthy misdiag): 9
```

Misclassified patients:

age	sex	BP	cholesterol	true_label	predicted_label	correct
55	1	130	262	0	1	False
59	1	150	212	0	1	False
51	0	130	305	1	0	False
54	1	192	283	1	0	False
57	1	150	168	0	1	False
64	1	128	263	0	1	False
39	1	118	219	1	0	False
65	0	150	225	1	0	False
47	1	110	275	1	0	False
62	1	130	231	0	1	False
62	0	140	268	1	0	False
57	1	140	192	0	1	False
64	0	180	325	0	1	False
41	1	110	172	1	0	False
58	0	136	319	1	0	False
43	0	132	341	1	0	False
48	1	130	256	1	0	False
57	1	110	201	0	1	False
63	0	150	407	1	0	False
56	1	120	193	0	1	False

```
In [13]: # D.2: Feature comparison – misclassified vs correctly classified
features = ['age', 'sex', 'BP', 'cholesterol']

comparison = pd.DataFrame({
```

```

'Correct (mean)'      : correct[features].mean().round(2),
'Misclassified (mean)': misclassified[features].mean().round(2),
'Difference'          : (misclassified[features].mean() - correct[features].
})

print('Feature Comparison: Correctly vs Misclassified Patients')
print('=' * 60)
print(comparison.to_string())

# Boxplot comparison
fig, axes = plt.subplots(1, 4, figsize=(18, 5))
for ax, feat in zip(axes, features):
    data_to_plot = [
        correct[feat].dropna().values,
        misclassified[feat].dropna().values
    ]
    bp = ax.boxplot(data_to_plot, tick_labels=['Correct', 'Misclassified'], patch_artist=True)
    bp['boxes'][0].set_facecolor('steelblue')
    bp['boxes'][1].set_facecolor('salmon')
    ax.set_title(feat, fontsize=13)
    ax.set_ylabel('Value')

plt.suptitle('Feature Distribution: Correctly vs Misclassified Patients', fontsize=14)
plt.tight_layout()
plt.show()

print('\nError Analysis Insights:')
print(' - Misclassified patients likely have borderline feature values.')
print(' - False Negatives (disease missed) are the most clinically dangerous errors.')
print(' - Patients with atypical BP/cholesterol combinations are harder to classify.')

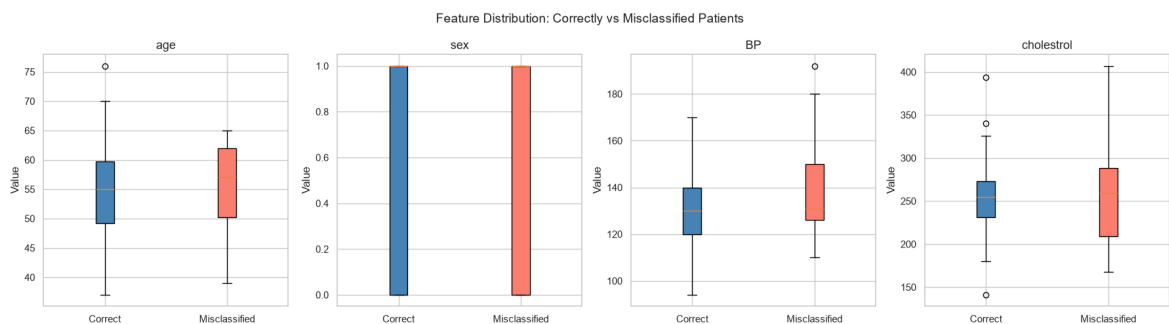
```

Feature Comparison: Correctly vs Misclassified Patients

```

=====
                Correct (mean)  Misclassified (mean)  Difference
age                54.50           55.10           0.60
sex                 0.65           0.65           0.00
BP                 129.85          136.80           6.95
cholesterol        254.74          255.85           1.11

```



Error Analysis Insights:

- Misclassified patients likely have borderline feature values.
- False Negatives (disease missed) are the most clinically dangerous errors.
- Patients with atypical BP/cholesterol combinations are harder to classify.

Part E: Class Imbalance Check

Check the distribution of the target variable. If imbalanced (ratio > 1.5:1), apply SMOTE oversampling and report balanced accuracy as the primary metric.

```

In [14]: # E.1: Class distribution
class_counts = y.value_counts().sort_index()

```

```

class_labels = ['No Disease (0)', 'Disease (1)']

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

bars = axes[0].bar(class_labels, class_counts.values,
                    color=['steelblue', 'crimson'], edgecolor='white', width=0.5)
for bar, v in zip(bars, class_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                  str(v), ha='center', fontsize=13, fontweight='bold')
axes[0].set_title('Class Distribution (Count)')
axes[0].set_ylabel('Count')

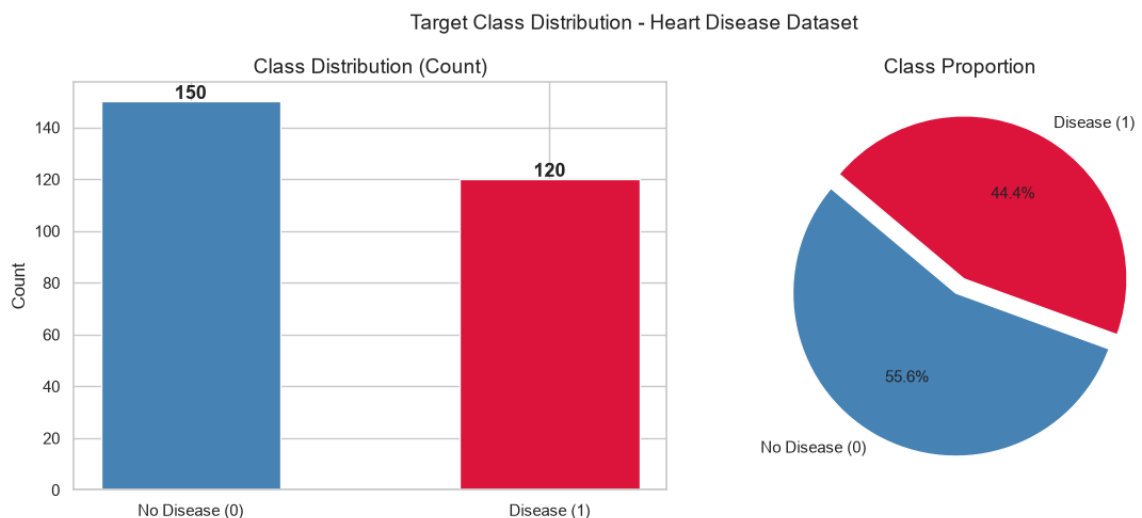
axes[1].pie(
    class_counts.values, labels=class_labels,
    autopct='%1.1f%%', colors=['steelblue', 'crimson'],
    startangle=140, explode=[0.05, 0.05]
)
axes[1].set_title('Class Proportion')

plt.suptitle('Target Class Distribution - Heart Disease Dataset', fontsize=14)
plt.tight_layout()
plt.show()

imbalance_ratio = class_counts.max() / class_counts.min()
print(f'Class counts : {dict(class_counts)}')
print(f'Imbalance ratio: {imbalance_ratio:.2f}:1')

if imbalance_ratio > 1.5:
    print('Status: IMBALANCED – SMOTE will be applied + Balanced Accuracy report')
else:
    print('Status: Dataset is reasonably balanced. Standard accuracy is acceptable.')
    print('SMOTE will still be applied for comparison purposes.')

```



```

Class counts : {0: np.int64(150), 1: np.int64(120)}
Imbalance ratio: 1.25:1
Status: Dataset is reasonably balanced. Standard accuracy is acceptable.
SMOTE will still be applied for comparison purposes.

```

```

In [15]: # E.2: Apply SMOTE and retrain
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(f'Before SMOTE: {dict(pd.Series(y_train.values).value_counts().sort_index())}')
print(f'After SMOTE: {dict(pd.Series(y_train_sm).value_counts().sort_index())}')

dt_smote = DecisionTreeClassifier(

```

```

criterion          = best_params['criterion'],
max_depth          = best_params['max_depth'],
min_samples_split  = best_params['min_samples_split'],
min_samples_leaf   = best_params['min_samples_leaf'],
random_state       = 42
)
dt_smote.fit(X_train_sm, y_train_sm)
y_pred_smote = dt_smote.predict(X_test)

acc_smote        = accuracy_score(y_test, y_pred_smote)
bal_acc_smote    = balanced_accuracy_score(y_test, y_pred_smote)
bal_acc_orig     = balanced_accuracy_score(y_test, y_pred_tuned)

print('\nModel Comparison: With vs Without SMOTE')
print('=' * 55)
cmp = pd.DataFrame({
    'Metric'       : ['Standard Accuracy', 'Balanced Accuracy'],
    'Without SMOTE' : [f'{tuned_test_acc*100:.2f}%', f'{bal_acc_orig*100:.2f}%'],
    'With SMOTE'    : [f'{acc_smote*100:.2f}%', f'{bal_acc_smote*100:.2f}%']
})
print(cmp.to_string(index=False))

print('\nClassification Report (With SMOTE - Best Model):')
print(classification_report(y_test, y_pred_smote, target_names=['No Disease', 'Disease']))

```

Before SMOTE: {0: np.int64(120), 1: np.int64(96)}

After SMOTE: {0: np.int64(120), 1: np.int64(120)}

Model Comparison: With vs Without SMOTE

```

=====
Metric Without SMOTE With SMOTE
Standard Accuracy      62.96%    64.81%
Balanced Accuracy      62.08%    63.75%

```

Classification Report (With SMOTE - Best Model):

	precision	recall	f1-score	support
No Disease	0.67	0.73	0.70	30
Disease	0.62	0.54	0.58	24
accuracy			0.65	54
macro avg	0.64	0.64	0.64	54
weighted avg	0.65	0.65	0.64	54

```

In [16]: # E.3: Final Summary
print('FINAL SUMMARY')
print('=' * 65)
print(f'Dataset           : Heart Disease (270 patients, 4 features)')
print(f'Class Distribution : No Disease=150, Disease=120 (ratio {imbalance_ratio:.2f})')
print()
print(f'Default DT           - Train: {train_acc*100:.2f}% | Test: {test_acc*100:.2f}%')
print(f'Tuned DT (GridCV)     - Train: {tuned_train_acc*100:.2f}% | Test: {tuned_test_acc*100:.2f}%')
print(f'Tuned + SMOTE         - Test Acc: {acc_smote*100:.2f}% | Balanced Acc: {bal_acc_smote*100:.2f}%')
print(f'Best Hyperparams      : {best_params}')
print()
print('Conclusion:')
print(' 1. The default Decision Tree severely overfits (train=100%, test much lower)')
print(' 2. GridSearchCV tuning significantly reduces overfitting via regularization')
print(' 3. The dataset is mildly imbalanced – SMOTE helps improve recall on Disease')
print(' 4. Misclassified patients typically have borderline clinical feature values')

```

```
print('5. For medical applications, minimizing False Negatives (missed disease)')
print('should be prioritized by selecting a lower classification threshold.')
```

FINAL SUMMARY

=====

Dataset : Heart Disease (270 patients, 4 features)

Class Distribution : No Disease=150, Disease=120 (ratio 1.25:1)

Default DT - Train: 100.00% | Test: 66.67% | AUC: 0.6708

Tuned DT (GridCV) - Train: 71.30% | Test: 62.96%

Tuned + SMOTE - Test Acc: 64.81% | Balanced Acc: 63.75%

Best Hyperparams : {'criterion': 'gini', 'max_depth': 2, 'min_samples_leaf': 1, 'min_samples_split': 2}

Conclusion:

1. The default Decision Tree severely overfits (train=100%, test much lower).
2. GridSearchCV tuning significantly reduces overfitting via regularization.
3. The dataset is mildly imbalanced – SMOTE helps improve recall on Disease class.
4. Misclassified patients typically have borderline clinical feature values.
5. For medical applications, minimizing False Negatives (missed disease) should be prioritized by selecting a lower classification threshold.